

Final Project Report

FounderRadar: Startup Ecosystem Intelligence Dashboard

GitHub: <https://github.com/khanhngoo/FounderRadar>

COMP4010 - Data Visualization

Nguyen Anh Duc (V202401631)
Ngo Dinh Khanh (V202401390)
Nguyen Dai Nghia (V202200779)
Lai Duc Minh (V202301105)

Under the supervision of
Prof. Huynh Thanh Trung



VINUNIVERSITY

College of Engineering and Computer Science
VinUniversity

June, 2026

1 Project Goal and Product Framing

FounderRadar is a Python Shiny dashboard for startup ecosystem research. It is designed for analysts who need to compare startup opportunity signals across companies, sectors, geographies, founder/team context, funding maturity, and historical outcomes. The dashboard does not claim to make investment decisions. It provides an auditable research workflow: find a pattern, filter the population, inspect the companies or model inputs behind the pattern, and decide whether the signal deserves further review.

The application has three connected pages. The **YC Ecosystem** page focuses on Y Combinator company discovery using radar scoring, hiring momentum, sector concentration, geography, founder background records, and text-derived topic clusters. The **Startup Intelligence** page studies historical startup outcomes using acquisition or IPO evidence as an exit proxy. The **ML Exit Predictor** page exposes the trained historical exit model through scenario controls, benchmark cards, a prediction distribution, global model signals, and a feature-value table.

The core design decision is to avoid a single black-box answer. A high YC radar score is a heuristic discovery signal, while a high startup exit probability is a model-derived historical ranking signal. Both are useful for triage, but neither should be read as proof of investment quality. FounderRadar is strongest when the user treats the charts as a starting point and uses the tables, hover details, and model inputs to audit what is driving each result.

This framing also shaped what was left out. The report and dashboard do not over-emphasize decorative explanations, broad startup advice, or generic background about venture capital. More space is given to dataset construction, model behavior, and visualization intent because those are the parts that determine whether the dashboard is trustworthy. The result is an applied analytics product rather than a static presentation: each page has a specific analytical job, and the report explains how those jobs connect.

The intended audience is a student project evaluator or analyst who wants to understand both the dashboard interface and the reasoning behind it. For that reason, the report emphasizes four questions: what data is being used, how those datasets are transformed, what the model actually measures, and how the visual encodings help users compare uncertain startup signals. This structure mirrors the application workflow and makes the dashboard easier to defend as a data visualization project rather than only as a software demo.

2 Data Sources, Joins, and Engineered Metrics

FounderRadar uses local Kaggle datasets and intentionally does not silently fall back to sample data. This matters because the charts and model are only meaningful when they reflect the intended project datasets. The repository stays source-only; downloaded CSV files remain local under `data/raw/`.

The YC side combines three datasets: Y Combinator Directory, 2024 Y Combinator All Companies Dataset, and Y Combinator Jobs Enriched. The pipeline searches each downloaded CSV for company-like fields and YC-specific signals such as batch, status, industry, tags, one-liners, hiring fields, founder names, schools, and prior companies. Candidate company tables are concatenated, normalized into common columns, and deduplicated by slug. Founder records are normalized separately and connected to companies through company slug or company-name-derived slug. When founder education and prior-company tables are available, they are summarized by founder handle and attached back to company context.

The Startup Intelligence side uses the relational Startup Investments dataset. The main company table comes from `objects.csv`. Acquisition and IPO files are used to define `success_exit`. Founder-role relationships are extracted from `relationships.csv`; education features come from `degrees.csv`; funding-round amount, participant, and timing features come from `funding_rounds.csv`. These tables connect through company object IDs and person object IDs, then collapse into one company-level modeling table.

The current local load contains 6,389 YC companies. For the Startup Investments data, the modeling table contains 42,751 rows with 3,250 recorded acquisition/IPO exits. This gives the project enough scale for exploratory visualization, while still requiring careful interpretation because all results depend on dataset coverage and historical record quality.

The two sides of the product are connected conceptually rather than by a direct row-level join. YC data describes a current accelerator ecosystem, while Startup Investments describes a broader historical startup population. Joining them directly would create false precision because the datasets do not share stable identifiers for every company. Instead, FounderRadar uses a parallel design: the YC page computes present-tense opportunity heuristics, the Startup Intelligence page computes historical outcome rates and model scores, and the ML page turns historical patterns into scenario-level comparisons. This keeps the source assumptions visible and avoids pretending that all datasets are one perfectly merged database.

Source area	How records connect	Metrics computed
YC companies	Concatenate company-like CSVs; normalize names, slugs, batches, status, industries, regions, tags, and hiring fields; deduplicate by slug.	Batch year/era, sector momentum, geography novelty, industry density, hiring signal, recognition score, momentum score, under-recognition signal, radar score, radar tier.
YC founder context	Normalize founder, school, and prior-company records; join to companies by company slug.	Top founder schools, top prior companies, founder-record counts in current filter.
Startup companies	Use <code>objects.csv</code> ; keep company entities; parse founding and funding dates; join exit evidence.	Founded year, first/last funding year, funding span, funding totals, funding rounds, milestones, relationships, success-exit flag.
Startup founders and rounds	Join founder relationships and degrees by person/company IDs; aggregate funding rounds by company ID.	Founder count, degree records, MBA/PhD/elite-school counts, prior-company links, observed rounds, total raised, median round, participant statistics, observed funding span.

Table 1: How the datasets are connected and transformed into dashboard metrics.

The engineered metrics are the bridge between raw CSV files and usable dashboard views. YC metrics translate sparse company records into a radar score based on recognition, momentum, geography novelty, industry density, hiring, batch timing, and team size. Startup metrics translate relational history into funding maturity, total raised, median round size, number of rounds, founder count, education indicators, prior-company links, and observed exit rates. The model adds predicted exit probability, dataset percentile position, and permutation-importance signals. These derived fields are what make the dashboard more useful than browsing the raw Kaggle tables directly.

A practical challenge is that the datasets have different levels of completeness. Some YC companies have rich founder and hiring signals, while others only have basic directory information. The Startup Investments records are broader but historical, so they contain more funding and exit history while being less aligned with the current YC ecosystem. FounderRadar handles this by exposing aggregate metrics and row-level tables together. If a chart shows a promising sector, the user can still inspect whether that signal is based on many companies, strong funding records, or a small number of unusually successful examples.

3 Model Design and Results

The historical exit model is implemented in `data_pipeline.py` as a scikit-learn pipeline. The input features include sector, country, founding year, funding totals, funding rounds, milestones, relationships, funding span, founder counts, education indicators, prior-company links, round amounts, participant counts, and observed round span. The preprocessing separates feature types: funding-like values are imputed and log-transformed, other numeric values are median-imputed, and categorical sector/country values are imputed and one-hot encoded.

The classifier is a balanced Random Forest with 70 trees and maximum depth 11. Model rows are capped through stratified sampling at 12,000 rows for performance. Evaluation uses a stratified train/test split with a 25 percent test set. ROC-AUC is the most relevant metric because the model is used to rank and compare scenarios, not to declare startup success as a hard binary decision.

The model also computes permutation importance using ROC-AUC drop when a feature is shuffled. Importance values are grouped into investor-readable families: Founder/team, Funding maturity, Market/geography, and Company metadata. This is important because a model dashboard should not only show a probability; it should also show which broad categories of evidence influence the model. In the ML Exit Predictor page, the user can adjust the scenario and immediately see the predicted exit probability, distribution position, top model signals, and exact feature values sent to the pipeline.

The model should still be interpreted conservatively. The target variable is an acquisition/IPO proxy, not a full definition of startup success. It omits revenue growth, private valuation changes, profitability, survival, and strategic value. It is also historical, so it reflects patterns in the dataset rather than guaranteed future outcomes.

The model results are therefore used as dashboard context, not as a final decision rule. The 0.832 ROC-AUC indicates that the model has useful ranking ability on the held-out split, but the 7.60 percent success-exit rate means accuracy can look high even when the positive class is rare. For that reason, the dashboard emphasizes relative position, upper-quartile benchmarks, feature-group importance, and exact feature values.

Metric	Value	Interpretation
Startup ML rows	42,751	Companies with enough signal to enter modeling.
Training sample cap	12,000	Performance cap before fitting the Random Forest.
Success exits	3,250	Acquisition or IPO proxy positives.
Success-exit rate	7.60%	Shows the class imbalance handled by balancing.
Held-out accuracy	0.871	Useful sanity check, but less informative under class imbalance.
Held-out ROC-AUC	0.832	Main ranking-quality signal for the model.
Dataset median prediction	21.91%	Benchmark shown on the ML page.
Dataset upper-quartile prediction	37.07%	High-side benchmark for scenario comparison.

Table 2: Current model results and benchmark values from the loaded Startup Investments data.

Those elements make the model more inspectable than a standalone probability card.

This modeling approach fits the dashboard because the product is exploratory. A Random Forest can capture nonlinear interactions between funding maturity, geography, founder/team signals, and company metadata without requiring the report to define a hand-written scoring equation for every possible case. At the same time, the dashboard avoids hiding the model behind a single number: it reports the training context, success-rate baseline, held-out metrics, feature-group importance, and scenario values. This makes the model output suitable for comparison and triage, but still transparent enough for a reviewer to question.

4 Visualization and Dashboard Design

The interface is designed as a compact investor workspace. It uses an off-white background, graphite text, compact panels, YC orange as the primary accent, and teal, blue, and rose as secondary signal colors. The layout avoids a marketing-style presentation because the target user needs to scan, compare, filter, and inspect details repeatedly.

YC view	Chart type	Main encodings	Purpose
Opportunity Quadrant	Bubble scatter	X recognition, Y momentum, size team, color industry	Finds companies with lower recognition and higher operating momentum.
YC Batch Pulse	Stacked streamgraph	X batch year, Y company count, color industry	Shows how YC sector concentration changes over time.
Industry x Region Matrix	Heatmap	X region, Y industry, color average radar score	Locates sector-geography pockets with stronger radar scores.
Radar Tier Distribution	Bar chart	X radar tier, Y company count, color tier	Summarizes the filtered opportunity pool.
Sector Share	Treemap	Tile size company count, color share percentage	Shows current YC sector concentration without long tables.
Founder Schools / Prior Companies	Horizontal bars	X founder-record count, Y school or company	Surfaces available founder background context.
Topic Projection Map	Bubble scatter	X/Y PCA topic coordinates, size team, color tier	Reveals thematic clusters from industry, tags, and one-liners.

Table 3: YC Ecosystem visualizations, encodings, and analytical role.

The visualization system uses shape and size deliberately. Scatterplots support tradeoff analysis; heatmaps support concentration scanning; bars support ranked comparisons; and tables support row-level inspection. Bubble size is used only when scale matters, such as team size, company count, or median funding. This helps prevent the user from over-reading a small segment with a high rate as if it were as reliable as a large segment. The Sector Risk / Reward chart is a good example of this design logic. The x-axis shows the number of companies in a sector, the y-axis shows observed exit rate, and dot size represents median funding. That means

Startup / ML view	Chart type	Main encodings	Purpose
Model Signal Importance	Heatmap	X signal group, Y feature, color ROC-AUC drop	Explains which feature groups move the exit model.
Funding Maturity Ladder	Horizontal bars	X exit rate, Y funding segment, color median funding	Compares observed exits across funding maturity stages.
Founder Signal Summary	Bubble scatter	X exit rate, Y signal segment, size company count, color dimension	Compares observed outcomes for team, education, and funding segments.
Sector Risk / Reward	Bubble scatter	X companies, Y exit rate, size median funding, color sector	Separates sector popularity, outcomes, and capital intensity.
Geography Ecosystem View	Bubble scatter	X companies log scale, Y exit rate, size median funding, label country	Compares countries by scale, outcome rate, and funding intensity.
Prediction Distribution	Histogram	X predicted probability, Y company count, vertical scenario marker	Shows where the active ML scenario lands in the dataset.

Table 4: Startup Intelligence and ML visualizations, encodings, and analytical role.

the chart separates three questions that are often mixed together: how common the sector is, how often exits are observed historically, and how capital-intensive the sector appears. Color identifies sectors, while hover details expose the exact counts and rates. Similar logic applies to geography and founder-signal charts: rates are useful, but sample size and funding context determine how much confidence a user should place in them.

Several charts intentionally combine aggregate views with drill-down tables. The aggregate views help the user identify patterns quickly, while the tables make the analysis auditable by exposing company names, one-liners, website links, funding totals, radar tiers, and model scores. This is especially important for a project using public startup data, where missing coverage can create misleading impressions if the user only sees a polished chart.

The table inventory also shows that the dashboard does not rely on one chart type repeatedly. Bubble scatterplots are used where there are tradeoffs between scale and outcome; heatmaps are used for two-dimensional categorical concentration; bar charts are used for rankings; treemaps are used for share of a filtered population; and histograms are used to show model-score distribution. This variety is intentional: each chart matches a specific analytical question instead of being chosen only for visual variety.

5 Interactivity and User Workflow

The dashboard workflow is iterative. A user begins with KPI cards and aggregate charts, filters the data, and then inspects companies or scenario inputs. The YC page supports filtering by batch era, industry, region, status, radar tier, and free-text search across company names, industries, regions, tags, and one-liners. This lets a user ask questions such as which recent YC companies in a sector have high radar scores, or which industries cluster in specific regions.

Startup Intelligence supports filtering by sector, country, minimum predicted exit probability, and text search. The minimum-probability slider is especially useful because it turns the model output into an exploratory threshold. As the threshold changes, the user can see which sectors, countries, founder segments, and funding stages remain in the filtered population.

The ML Exit Predictor makes the model more concrete. Instead of only reading global model importance, a user can define a hypothetical company by sector, country, founding year, funding totals, funding rounds, milestones, relationships, founder count, education records, prior-company links, participant counts, and funding span. The page then shows predicted exit probability, dataset median, upper quartile, the scenario marker on the prediction distribution, top model signals, and the exact feature-value table.

This workflow supports several realistic research questions:

- Which YC companies have high momentum but remain relatively less recognized?
- Which sectors have both large company count and stronger observed exit outcomes?
- Which geographies combine startup scale, exit rate, and capital intensity?
- How do founder/team records or funding maturity segments compare on observed exit rate?
- How does a hypothetical company profile compare with historical model-score benchmarks?

Plotly hover details and Shiny data grids make the analysis auditable. Users can inspect company names, one-liners, radar tiers, exit rates, median funding, and company counts without overcrowding the charts. The key product idea is that aggregate charts reveal where to look, while tables and scenario feature values explain what is behind the visual pattern.

This workflow also explains why several metrics are recomputed after filters are applied. KPI cards, chart rates, sector summaries, geography summaries, and company tables are not static screenshots of the original dataset. They respond to the active filters, which lets the user compare the full market with a narrowed research subset. This is especially important for the probability threshold on the Startup Intelligence page: moving the slider changes not only the company table, but also the distribution of sectors, countries, funding stages, and founder/team signals that remain visible.

From an interaction-design perspective, the dashboard follows an overview-first pattern. KPI cards establish scale, charts identify patterns, hover details explain local points, and tables support verification. The ML page uses the same pattern at a scenario level: a user sets feature values, reads the predicted probability, compares it to dataset benchmarks, and then checks the feature table to confirm what was sent to the model. This repeated structure lowers cognitive load because the same analysis loop appears across all three pages.

6 Limitations, Ethics, and Future Work

FounderRadar has several important limitations. The Kaggle datasets may be incomplete, stale, or unevenly covered. Founder school and prior-company data are especially sensitive to source coverage, so missing records should not be interpreted as weak founders. Historical sector labels may not map cleanly to modern startup categories, and country-level aggregation can hide city-level ecosystem differences.

The model target is also limited. Acquisition or IPO evidence is a practical proxy, but it excludes revenue growth, profitability, long-term survival, private valuation increases, employment impact, and strategic importance. For this reason, the predicted probability should be read as a historical ranking aid rather than a direct forecast. Founder education features also introduce possible prestige bias and coverage bias, which should be stated clearly when presenting the dashboard.

There are technical limitations as well. Data loading and model fitting happen at app import time, which can slow startup. Plotly is loaded from a CDN, so full chart rendering depends on network access. The YC Company Explorer caps displayed rows for performance, and model fitting plus permutation importance are sampled or capped. These are acceptable for the current project but would need refinement in a production deployment.

The most valuable future improvements are methodology notes inside the UI, calibration and reliability charts for the model, a confusion matrix at the selected probability threshold, CSV export for filtered tables, and sample-size labels on rate-based charts. Another useful extension would be persistence for filter states, so a user can save a research screen and return to the same sector, geography, or model scenario later. Accessibility can also improve through keyboard-visible focus states, text summaries below complex charts, and encodings that do not rely only on color.

For a future production version, model training should also move out of the app import path. The current implementation is acceptable for a project because it keeps the pipeline self-contained and reproducible, but startup latency would matter in a deployed dashboard. A scheduled preprocessing step or cached model artifact would make the application faster while preserving the same visual workflow.

FounderRadar's main contribution is not perfect prediction. Its contribution is a coherent, auditable workflow for comparing startup signals across current YC ecosystem data and historical startup outcome data. It connects scoring, charts, filters, model transparency, and company-level inspection so users can discover patterns while still respecting uncertainty.